Measuring digitizer delay with a square wave

Test date: 5 November 2026 Saved-data analysis using a fitted measured curve: 12 November 2026

Purpose and result

The delay through the ADS127L18 digitizer and its output voltage translator was measured using a square-wave input. The input transitions were timestamped directly by the ESP32-P4. A measured output transition was reconstructed by combining ADC readings from repeated input edges. A smooth curve was fitted to those readings, and its halfway crossing was used to calculate the delay.

Rising-edge delays of approximately **2535 ns** were obtained at both 997 Hz and 10 kHz, after the separately measured difference between the two P4 input timings was subtracted. Five captures at each frequency were included. The result was defined by the halfway crossing of the fitted measured curve. A complete 95% absolute uncertainty bound was not established.

Connections

A square wave was supplied by the Rigol signal generator, with 50% duty cycle and nominal levels of 0 and 3.3 V. The signal was split into two paths with a resistor bridge:

Path	Connection	Purpose
Direct input	Generator to P4 GPIO6	The time of each input transition was recorded.
Digitizer input	Generator through a voltage divider to CH0P; CH0N grounded	The same transition was digitized at a reduced voltage.
Digitizer output	FSYNC, DCLK, and DOUT0 through the voltage translator to the P4	ADC values were decoded, and their associated FSYNC pulses were identified.

FSYNC was connected to GPIO1. Common grounds were connected. During setup, approximately 0.13 V and 1.46 V were observed at the loaded digitizer input. The initial and final levels were subsequently estimated separately for every recorded transition rather than assumed from the generator setting.

Samples and capture

ADC samples were produced at 1,024,000 per second, approximately 977 ns apart. Forty consecutive ADC readings surrounding each selected input transition were saved. Both rising and falling transitions were included.

The P4 hardware capture peripheral, MCPWM, was used to record the timer count when an edge was detected. The same 160 MHz timer was used for the direct input at GPIO6 and the translated FSYNC signal at GPIO1. Each timer tick represented 6.25 ns. The recorded edge time was therefore independent of when software subsequently read the register.

ADC readings, input-edge timestamps, and numbered FSYNC timing references were exported to the PC. The delay calculations were performed in Python from these saved records.

Assigning a time to each ADC reading

An ADC value and its output time were obtained separately. The value was decoded from DOUT0, using DCLK and FSYNC to identify its bits and sample number. Its output time was assigned to the beginning of that sample's

FSYNC pulse, as received by the P4 through the translator.

Not every FSYNC timestamp was saved. Instead, numbered FSYNC timing references were recorded across the capture. The average number of timer ticks per sample was calculated from the first and last references:

```
ticksPerSample = (lastTicks − firstTicks) / (lastSample − firstSample)
```

For a reading numbered i, its FSYNC time relative to the corresponding direct input edge was calculated as follows:

```
relativeTicks = (i − firstSample) × ticksPerSample
              − (inputEdgeTicks − firstTicks)

t[i] = relativeTicks × 6.25 ns
```

Elapsed timer counts were calculated with counter wraparound accounted for. Intermediate FSYNC references were checked against this relationship. Only input edges within the first-to-last reference interval were accepted.

Thus, zero on the time axis was the input transition detected at GPIO6. Positive times identified later FSYNC pulses. The roughly 977 ns spacing between FSYNC pulses did not prevent their position relative to the input edge from being determined with the finer timer ticks.

Reconstructing the measured transition

For each recorded transition, the initial voltage was calculated as the mean of the first eight ADC readings. The final voltage was calculated as the mean of the last eight. Each reading was then expressed as a fraction of the transition:

```
fraction[i] = (voltage[i] − vBefore) / (vAfter − vBefore)
```

The initial level was thereby represented by 0%, and the final level by 100%. Falling transitions were represented in the same direction, because both numerator and denominator were negative. Rising and falling results were kept separate.

The readings from all five captures at each frequency were combined. Each reading retained its time relative to its own GPIO6 input edge. No individual transition was shifted horizontally to improve agreement.

Because the input generator and ADC sampling clock were not synchronized, different portions of the transition were sampled on successive edges. By combining these readings, the gaps between the readings of any single edge were filled with actual measurements from other edges.

Calculating delay from the fitted curve

A smooth curve was fitted to the combined readings near 50%. A cubic polynomial was used: a curve with constant, linear, squared, and cubed terms. Its coefficients were calculated by least squares, minimizing the sum of squared differences between measured and fitted voltage fractions. Each selected reading was given equal weight.

Readings between 2050 and 3050 ns after the GPIO6 edge were selected. This interval surrounded the crossing found in the earlier analysis. Time was scaled for the calculation:

```
x = (t − 2550 ns) / 1000 ns

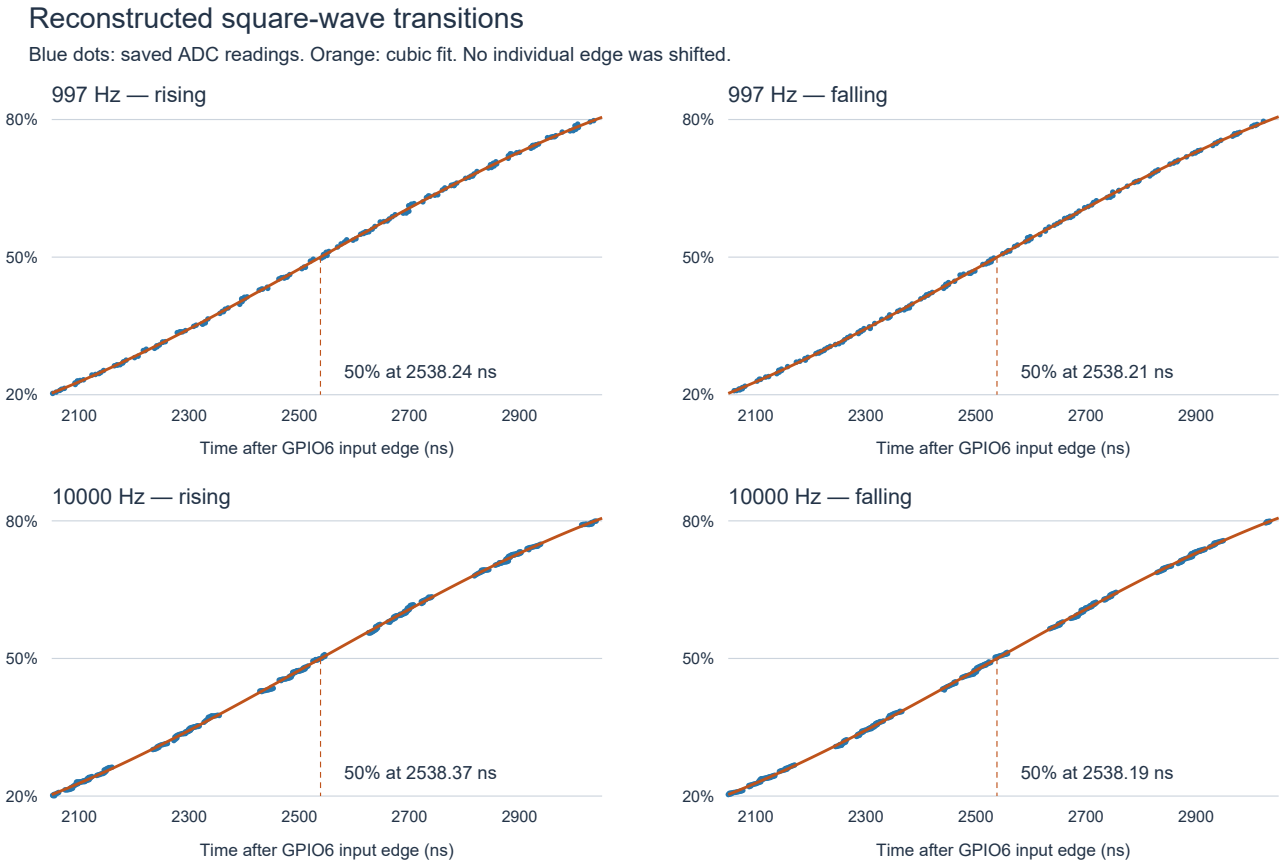
fitted fraction = 0.5 + a0 + a1 × x + a2 × x² + a3 × x³
```

The halfway crossing was obtained by solving:

$$a0 + a1 \times x + a2 \times x^2 + a3 \times x^3 = 0$$

$$tHalf = 2550\ ns + x \times 1000\ ns$$

Exactly one crossing within the fitted interval was required. Because each input edge had already been assigned time zero, `tHalf` was the measured delay before correction for the difference between P4 inputs. The value 2550 ns was used only to center the fitting interval and scale the arithmetic; the crossing was determined by the measured readings.



Falling transitions were normalized to 0–100%. P4 input timing correction was not subtracted.

The saved readings were shown as blue dots, and the fitted curves as orange lines. The uncorrected halfway crossings were marked. Only the neighborhood of the crossing was plotted. The input voltage waveform was not represented by these curves.

Why delay could be estimated more finely than 977 ns

Consecutive readings of any one transition were separated by approximately 977 ns. However, the readings from different transitions occurred at different positions relative to their input edges. Those positions were determined from the P4 timer, with 6.25 ns ticks, and the numbered FSYNC references.

Near the halfway crossing, the largest gap between the combined measurement times was approximately 22 ns at 997 Hz and 79 ns at 10 kHz. The transition was therefore described by substantially more measured points than were available from a single edge. Its halfway crossing was estimated from a curve fitted to those points, rather than from a straight line between just two readings.

This reconstruction depended on a repeatable transition shape. Noise and changes between transitions were reflected in the scatter of the points. Finer resolution than 977 ns was obtained, but an absolute accuracy bound was not established by the density of the points alone.

Results

The combined-curve crossings were obtained as follows. The transition counts described the saved edges contributing to each group; only readings inside the fitting interval were used for the local fit.

Input frequency	Transition	Number of transitions	Fitted halfway crossing, before P4 input correction
997 Hz	Rising	176	2538.24 ns
997 Hz	Falling	178	2538.21 ns
10 kHz	Rising	320	2538.37 ns
10 kHz	Falling	320	2538.19 ns

In a separate test, the same rising signal was supplied to GPIO1 and GPIO6. GPIO1 was recorded approximately 3.04 ns later than GPIO6. This difference was subtracted from the rising-edge results:

```
corrected delay = fitted halfway crossing − 3.04 ns
```

Input frequency	Corrected rising-edge delay
997 Hz	2535.20 ns
10 kHz	2535.33 ns

Thus, a rising-edge delay of approximately **2535 ns** was obtained at both frequencies. Extra decimal places were retained to show the calculations, not to claim accuracy to hundredths of a nanosecond. The rising-edge correction was not applied to the falling-edge results.

The input-to-output path, including the FSYNC voltage translator, was measured. A chip-only ADC delay was not isolated. The existing firmware correction was not changed by this reanalysis.

Checks and uncertainty

The saved captures were checked for correct sample-to-FSYNC association, acceptable ADC status bits, consecutive alternating input edges, and input frequency consistent with the generator setting. Settled initial and final levels and a unique halfway crossing were required for each saved transition.

The dependence on the fitted curve was examined by using quadratic, cubic, and fourth-degree polynomials, with fitting windows of ±350, ±500, ±700, and ±900 ns around 2550 ns. Uncorrected crossings across all four groups and these choices ranged from approximately **2533 to 2540 ns**. A sensitivity of a few nanoseconds was indicated; this range was not a 95% confidence interval.

The cubic fit was also repeated separately for each capture. Uncorrected rising-edge crossings ranged from 2536.35 to 2541.19 ns at 997 Hz and from 2535.24 to 2541.45 ns at 10 kHz. These ranges described variation between the five captures at each frequency.

Three limitations were retained:

- **A fast input transition was assumed.** The GPIO6 rising threshold had been measured at approximately 1.765 V, with a sample standard deviation of 0.025 V. However, the input edge waveform during the original calibration was not recorded. Its rise time was assumed negligible compared with the digitized transition, so no additional threshold-to-input-midpoint correction was applied.

- **The measured output curve was fitted, not known exactly.** Curvature was represented using many measured points, but some dependence on the fitting interval and curve choice remained. The applicability of the 3.04 ns P4 input correction also depended on the edge shapes used in the separate input comparison.

- **A complete absolute uncertainty bound was not established.** Capture-to-capture variation and fit sensitivity were examined. Input-path differences, timer resolution, and sample-time assignment also contributed uncertainty and were not fully quantified by those checks.

The result was tied specifically to the halfway point of the digitized square-wave transition. In the separate sine-wave test, delay was defined by the fitted phase shift; identical results from the two definitions were not assumed.

Code and retained evidence

- fitPulseTransitionCurve.py: normalized readings were combined, local curves were fitted, and halfway crossings and sensitivity checks were calculated.

- Curve-fitting results: combined fits, individual capture fits, measurement coverage, and alternative curve/window results were retained. Values in this file were recorded before the P4 input correction.

- pulseDelayReport.py: sample times were assigned by `analyze`; transition levels and normalized readings were obtained by `measureEdge`. The original analysis code and saved captures were not changed.

- 997 Hz capture results and 10 kHz capture results: the five trials, individual ADC transitions, and timing references were retained for each frequency.

- Sine-wave delay measurement: the independent phase-shift method and its results were documented separately.